

Dynamic-opposite learning enhanced meta-heuristic approach for solving multiple industrial optimization problems

Received: 13 July 2025

Accepted: 26 March 2026

Published online: 20 April 2026

Cite this article as: Wu H., Li L., Wang G. *et al.* Dynamic-opposite learning enhanced meta-heuristic approach for solving multiple industrial optimization problems. *Sci Rep* (2026). <https://doi.org/10.1038/s41598-026-46614-7>

Hao Wu, Lijuan Li, Guohui Wang, Zhile Yang & Binhong Shi

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

Dynamic-Opposite Learning Enhanced Meta-Heuristic Approach for Solving Multiple Industrial Optimization Problems

Hao Wu^{1,2}, Lijuan Li^{1,2,*}, Guohui Wang³, Zhile Yang⁴, and Binhong Shi⁵

¹School of Optoelectronic Engineering, Changchun University of Science and Technology, Changchun, Jilin 130022, China

²Zhongshan Institute of Changchun University of Science and Technology, Zhongshan, Guangdong 528400, China

³Liaoshen Industries Group Co., Ltd., Shenyang, 110000, China

⁴Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, Shenzhen, Guangdong 518000, China

⁵Institute of International Engineering, Shenyang Ligong University, Shenyang, 110000, China

*Corresponding author: Lijuan Li, custjuan@163.com

ABSTRACT

Advanced industrial manufacturing involves a number of complex operation and decision NP-hard problems, such as industrial robot trajectory planning and flexible job shop scheduling, which calls for effective optimization approaches. In this work, we propose an enhanced version of the moth-flame optimization (MFO) algorithm, referred to as dynamic-opposite learning differential evolution MFO (DOLDEMFO), tailored for addressing continuous and integrated industrial optimization challenges. A dynamic opposite learning (DOL) strategy is embedded to asymmetrically refine the search region, enhancing both exploration and exploitation potential. Additionally, differential evolution (DE) is incorporated into the MFO framework, leveraging its structural simplicity, convergence characteristics, and robustness, to further refine search efficiency. To evaluate the proposed approach, extensive experiments on CEC series benchmarks and two industrial cases were performed. Comparative analysis with conventional metaheuristics verifies that DOLDEMFO consistently delivers competitive results and consistent performance across diverse industrial optimization tasks.

1 Introduction

Moth-flame optimization (MFO), originally introduced by¹, is a bio-inspired computational technique that mimics the navigation mechanism of moths in nature. It has found broad applications in areas such as neural network training², numerical function minimization³, and diverse engineering tasks⁴. Li et al.⁵ introduced a dual-evolution MFO variant tailored for real-valued optimization. Buch et al.⁶ utilized the approach to tackle challenges in optimal power flow systems. Deb et al.⁷ investigated both MFO and Whale Optimization Algorithm (WOA) for efficient multilevel image thresholding. Yildiz et al.⁸ proposed an enhanced MFO version targeting manufacturing-related optimization. Additionally, Li et al.² developed a composite model combining MFO with least squares support vector machine (LSSVM) to predict annual power load demands.

Beyond the standard MFO, numerous enhanced versions have been developed to improve its performance. For example, Li et al.⁵ introduced a dual-evolution framework for MFO, incorporating flame generation via differential evolution and a dynamic mechanism to steer the search behavior of moths. Jia et al.⁹ presented the SAMFO-TH model for image segmentation, which integrates an adaptive inertia adjustment mechanism and a novel thresholding approach to balance local exploitation and global exploration. Luo et al.¹⁰ combined grey wolf optimization with MFO initialization, resulting in a hybrid method named GWOMFO. Xu et al.¹¹ combined MFO with extreme learning machine to predict mice's phenanthrene poisoning. Si et al.¹² combined MFO and fruit fly optimization algorithm which is called MOF-FOA to obtain the optimal network parameters for radical basis function neural network. Ji et al.¹³ combined binary gravitational search algorithm inspired by quantum and modified MFO to solve the co-scheduling of cascaded locks with multiple chambers problem (CCLM). Sapre et al.¹⁴ used opposition-based learning, Cauchy mutation and evolution boundary constraint handling to modify MFO to improve its exploration capability. Although the existing methods have improved MFO a lot, it still has large probability to be trapped with local optimum.

Robot trajectory optimization is inherently challenging due to the nonlinear nature of motion paths and the requirements for high efficiency, smooth dynamics, and low acoustic emissions in industrial settings¹⁵. Numerous studies have considered

objectives such as reducing motion duration, minimizing acceleration profiles¹⁶, and lowering energy expenditure during path planning^{17,18}. In recent years, hybrid optimization strategies have gained traction in robotic research. For instance, Zhang et al.¹⁹ introduced a multi-objective framework to address path planning in environments with obstacles, targeting both time and acceleration efficiency. Their method leverages quintic B-splines to construct joint-space trajectories under combined motion and obstacle constraints. Similarly, Wang et al.²⁰ developed an approach to shape objective functions based on temporal and velocity considerations. A variety of hybrid algorithms have also been explored, such as IWOA-PSO (an improved whale algorithm integrated with particle swarm optimization)²¹, EDOLSCA (elite and dynamic opposite learning enhanced sine cosine algorithm)²², WOA-GA (whale optimization coupled with genetic algorithm)²⁰, and HGAPSO (a hybrid of genetic algorithm and adaptive particle swarm optimization)²³, all aiming to enhance the balance between global exploration and local exploitation.

One notable challenge in industrial optimization is the flexible job-shop scheduling (FJSP) problem. FJSP involves assigning processing tasks to appropriate machines and determining their execution order under specified constraints, with the objective of minimizing factors such as total production time, energy usage, and other operational costs²⁴. Over the years, a wide range of solution strategies have been explored, primarily falling into two categories: exact mathematical programming techniques^{25–27} and heuristic-based meta-optimization methods (MAs)^{28–33}. The former approaches adopt mathematical programming rules and the later ones are belonging to artificial intelligence based approaches, dispatching rules (DRs) and local search. For deterministic mathematical programming methods, Demir et al.³⁴ compiled some mathematical models from other literature and also proposed a time-indexed model for solving FJSP, and they formulated different models considering different constraints for minimizing the makespan. Roshanaei et al.³⁵ presented two mixed-integer linear programming models to solve large size FJSP and compared the results with other seven out-performing MAs. However, deterministic mathematical programming methods do not perform well for large scale FJSP^{36,36,37}. Baykasoğlu et al.^{38–42} presented the effects of DRs with different levels of flexibility. Tay et al.⁴³ used genetic programming along with DRs to solve FJSP. Gao et al.⁴⁴ proposed a two-stage artificial bee colony algorithm with new job insertion. For local search methods, Jia et al.⁴⁵ presented a new algorithm based on the tabu search algorithm along with back-jump tracking. Kemmoe-Tchomte et al. proposed a multi-level evolutionary local search paradigm and a procedure to accelerate the estimation of the neighborhood quality for local search phases⁴⁶. Ho et al. combined the evolutionary algorithm and guided local search to solve multiple-objective FJSP^{47–51}. In the domain of meta-heuristic algorithms (MAs), several researchers have introduced novel frameworks based on tabu search integrated with population-based techniques^{52–58}. Driss et al.⁵⁹ developed a genetic algorithm enhanced by innovative crossover and mutation schemes. Nouri et al.⁶⁰ employed a PSO-based distribution mechanism to tackle the dynamic FJSP variant. Xu et al.⁶¹ incorporated TLBO (teaching–learning-based optimization) with complementary strategies to improve the balance between global search and local refinement. Yuan et al.⁶² integrated DE (differential evolution) with other techniques to construct a hybrid solver tailored for FJSP. These investigations collectively indicate that MAs generally outperform exact optimization methods in handling FJSP. Nonetheless, the inherent complexity of FJSP still necessitates advanced computational solutions to further elevate optimization efficiency.

In this research, we propose an enhanced hybrid framework that integrates the dynamic-opposite learning (DOL) mechanism⁶³ with differential evolution (DE) to substantially improve the optimization performance of the original Moth-Flame Optimization (MFO) algorithm. The DOL strategy introduces an adaptive and non-symmetric search landscape, which facilitates the discovery of high-quality solutions. Meanwhile, DE—acknowledged for its structural simplicity, ease of implementation, rapid convergence characteristics, and robustness—is employed to reinforce the exploratory capabilities of the base algorithm⁶⁴. Moreover, a scaling factor is incorporated to dynamically adjust the search boundaries, thereby achieving a more effective balance between exploration and exploitation throughout the optimization process within the proposed DOLDEMFO framework.

The principal contributions of this study are summarized as follows:

1. A refined variant, denoted as DOLDEMFO, is developed by embedding the DOL mechanism and DE operator into the conventional MFO framework, thereby enhancing its search efficiency and solution quality.
2. A series of numerical experiments on standard benchmark functions are conducted to comprehensively assess the performance and validate the effectiveness of the proposed algorithm.
3. Two application-driven optimization cases—namely the Flexible Job Shop Scheduling Problem (FJSP) and Trajectory Optimization—are formulated using detailed mathematical models and constraints, serving as practical validations of the algorithm's applicability.

The remainder of this paper is structured as follows: Section 2, Algorithm Preliminaries, introduces the theoretical background including the moth-flame optimization technique, dynamic-opposite learning, and differential evolution. Section 3, DOL and DE based MFO algorithm, details the proposed DOLDEMFO approach. Section 4, Experimental results and

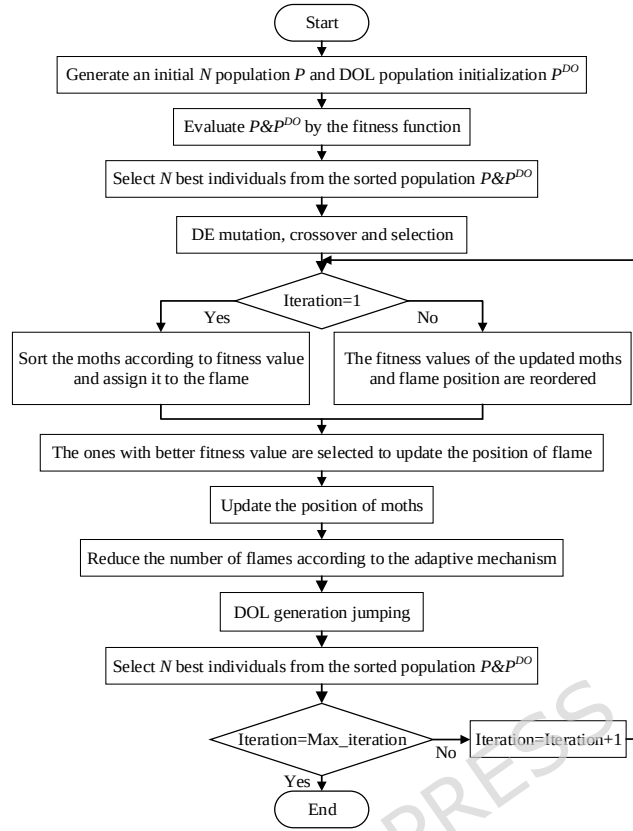


Figure 1. The flowchart of the proposed DOLDEMFO

discussions, presents comprehensive benchmarking and performance analysis. Section 5, Industrial Optimization Problem Applications, demonstrates the algorithm's effectiveness on optimal trajectory planning and the flexible job-shop scheduling problem. Section 6 concludes the paper.

2 Algorithm Preliminaries

2.1 Overview of the Moth-Flame Optimization Technique

The Moth-Flame Optimization (MFO) algorithm, introduced in 2015¹, belongs to the class of population-based metaheuristics. It is conceptually grounded in the natural flight behavior of moths, which exhibit a tendency to navigate by maintaining a fixed angle relative to distant light sources, often resulting in a spiral movement pattern around flames during nighttime.

- In the context of MFO, each moth represents a candidate solution, and the collective movement of moths explores the defined search domain. The collection of all reachable positions forms the **search space**, where the position associated with the highest fitness is regarded as the **optimal solution**.
- The role of the moth is to constantly update its movement and eventually converge to the optimal position.
- **Flames** are used to preserve the best solutions (positions) found by moths during the search process.
- During each iteration:
 - Each moth searches around its current best-known position.
 - If a better solution is found, it replaces the current optimal position stored in the flame.
 - Moths update their positions based on a logarithmic **spiral function** to simulate the natural flying behavior.
- The iteration continues until a stopping condition is satisfied, such as the maximum number of iterations or convergence.

Algorithm 1 The DOLDEMFO Optimization Procedure

```

1: Initialize the population  $P$  using random sampling;
2: for  $i = 1$  to  $N$  do
3:   Generate  $r1_i, r2_i$  from  $U(0, 1)$ ;
4:   for  $j = 1$  to  $D$  do
5:      $P_{ij}^{DO} \leftarrow P_{ij} + r1_i \cdot (r2_i \cdot (a_j + b_j - P_{ij}) - P_{ij})$ ;
6:     Enforce boundary constraints;
7:   end for
8: end for
9: Select  $N$  most fit individuals from  $P \cup P^{DO}$ ;
10: Set generation counter  $G \leftarrow 0$ ;
11: while  $G < \text{MaxIterations}$  do
12:   Compute fitness values for all individuals via  $f(\cdot)$ ;
13:   for  $i = 1$  to  $N$  do
14:     Perform mutation using equation (7);
15:     Conduct crossover as defined in equation (8);
16:     Execute selection based on equation (9);
17:   end for
18:   Update fitness values using  $f(\cdot)$ ;
19:   if  $G = 1$  then
20:     Rank individuals by fitness to initialize bestflames;
21:   else
22:     Merge current population with previous bestflames;
23:     Sort merged set by fitness;
24:     Retain top  $N$  individuals as updated bestflames;
25:   end if
26:   for  $i = 1$  to  $N$  do
27:     Use rule (1) to update candidate  $p$ ;
28:     Apply boundary handling;
29:     Evaluate  $f(p')$  for the newly generated solution  $p'$ ;
30:     if  $f(p) < f(p')$  then
31:       Substitute  $p$  with  $p'$ ;
32:     end if
33:   end for
34:   Increment  $G \leftarrow G + 1$ ;
35:   if  $\text{rand} \leq J_r$  then
36:     for  $i = 1$  to  $N$  do
37:       Sample  $r3_i, r4_i$  from  $U(0, 1)$ ;
38:       for  $j = 1$  to  $D$  do
39:          $a_j \leftarrow \min(P_{ij}), b_j \leftarrow \max(P_{ij})$ ;
40:          $P_{ij}^{DO} \leftarrow P_{ij} + w \cdot r3_i \cdot (r4_i \cdot (a_j + b_j - P_{ij}) - P_{ij})$ ;
41:         Apply boundary check;
42:       end for
43:     end for
44:     Re-select  $N$  fittest individuals from  $P \cup P^{DO}$ ;
45:      $G \leftarrow G + 1$ ;
46:   end if
47: end while

```

The position update rule for each moth is formulated as:

$$M_i = S(M_i, F_j) \quad (1)$$

Here, the function $S(M_i, F_j)$ governs the trajectory by which a moth M_i moves toward a target flame F_j , and is defined as:

$$S(M_i, F_j) = D_i e^{bt} \cos(2\pi t) + F_j \quad (2)$$

In this context, D_i denotes the Euclidean distance between the i^{th} moth and the j^{th} flame, b represents the constant that determines the shape of the logarithmic spiral, and t is a randomized coefficient sampled uniformly from the interval $[-1, 1]$. The distance metric D_i is explicitly calculated as:

$$D_i = |F_j - M_i| \quad (3)$$

To effectively regulate the exploration-exploitation trade-off throughout the optimization process, a dynamic strategy is adopted to decrease the number of active flames over successive iterations. Moths associated with eliminated flames subsequently align their positions with the flame exhibiting the poorest objective value. The reduction rule for the flame count at a given iteration is given by:

$$flame.number = round\left(N - G \cdot \frac{N-1}{G_m}\right) \quad (4)$$

where G represents the current iteration number, N is the starting number of flames, and G_m denotes the total number of iterations in the optimization process.

2.2 Dynamic-opposite learning

Numerous OBL variants have been developed, such as quasi-opposite based learning (QOBL)⁶⁵ and quasi-reflection based learning (QRBL)^{66,67}, both of which demonstrate improved convergence towards the global optimum compared to the original OBL. Nevertheless, these approaches are less effective when a local optimum exists within the search space. The fundamental concept of OBL is to extend the search space by generating the opposite of a candidate solution, thereby increasing the likelihood of approaching the global optimum. However, due to the static definition of positions, OBL still tends to converge to local optima. In response to this limitation, the DOL strategy was introduced in⁶³.

In the DOL strategy, a random opposite number X^{RO} is proposed as the opposite number X^O multiplying a random value in $[0, 1]$. This is the key mechanism that introduces asymmetry, in stark contrast to the standard OBL where the opposite point is always symmetrically located with respect to the center of the search interval. By changing X^O into X^{RO} , the searching space becomes asymmetric, facilitating the improvement of the exploration capability. This asymmetry is crucial: instead of jumping to a fixed, mirrored point, the search agent now explores a randomized point (X^{DO}) in the direction of this asymmetrically-scaled opposite point (X^{RO}). This dynamic and non-symmetrical jump effectively disrupts the convergence path towards a local optimum, providing a better chance to explore previously unvisited regions of the search space. The search space has three cases: X^{RO} is between X and X^O ; X^{RO} is larger than X^O ; X^{RO} is smaller than X . Then DOL is implemented by randomly selected a DOL number X^{DO} from X to X^{RO} . The DOL strategy makes the searching space shrink with the iteration increasing dynamically, leading to the enhancement of the exploitation capability. A positive weighting factor w is used to balance the exploitation and exploration capabilities. The mathematical model is showed as follows:

2.2.1 Dynamic Opposite Number

The dynamic opposite number is computed using the following equations:

$$\begin{aligned} X^{RO} &= rand \cdot X^O \\ X^{DO} &= X + w \cdot rand \cdot (X^{RO} - X) \end{aligned} \quad (5)$$

Here, X^{RO} represents the randomly scaled opposite value, while X^{DO} denotes the dynamically adjusted opposite value. The term $rand$ indicates a stochastic scalar sampled uniformly from the interval $(0, 1)$, and w is a positive control coefficient that adjusts the influence of the directional difference.

2.2.2 Dynamic Opposite Point

The formulation for computing the dynamic opposite point along each dimension is given by:

$$X_j^{DO} = X_j + w \cdot rand \cdot (rand \cdot X_j^O - X_j), \quad j = 1, \dots, D \quad (6)$$

In this formulation, X_j^{DO} refers to the j^{th} component of the dynamic opposite vector in a D -dimensional search space.

2.2.3 DOL based optimization

During each iteration, X^{DO} is generated based on X . The current solution X is replaced by X^{DO} if the latter exhibits superior fitness. Otherwise, X remains unchanged. Furthermore, each component of X^{DO} must lie within the range $[a_j, b_j]$; if not, it is reinitialized with a random value within the corresponding interval.

2.3 Differential evolution

The flow of DE is similar to that of GA, which uses a differential method in the process of population mutation, and can correspond any two random individuals to the individual to be optimized during the optimization. Compared with other algorithms, it reduces the complexity of the algorithm, the formulations are as follows:

$$v_i(G+1) = X_{r1}(G) + F * (X_{r2}(G) - X_{r3}(G)) \quad (7)$$

$$u_{j,i}(G+1) = \begin{cases} v_{j,i}(G+1) & \text{if } rand \leq CR \text{ or } j = j_{rand} \\ x_{j,i}(G) & \text{otherwise} \end{cases} \quad (8)$$

where $X_{r1}(G)$, $X_{r2}(G)$, and $X_{r3}(G)$ are three distinct individuals randomly selected from the population at generation G , with $r1 \neq r2 \neq r3 \neq i$. F is the scaling factor that controls the amplification of the differential variation. CR is the crossover rate, and j_{rand} is a randomly selected dimension index to ensure that the trial vector receives at least one component from the mutant vector.

Then comparing the fitness of the former individual with the one that is obtained by a series operation, choose the better one into next iteration:

$$x_i(G+1) = \begin{cases} u_i(G+1) & \text{if } f(u_i(G+1)) \leq f(x_i(G)) \\ x_i(G) & \text{otherwise} \end{cases} \quad (9)$$

3 DOL and DE based MFO algorithm

In this section, a novel DOL based MFO algorithm, named DOLMFO is proposed, where DOL is embedded with MFO to accelerate the convergence speed. DOL contributes to two steps of MFO, including population initialization and generation jumping. The detailed process are defined as follows.

3.1 DOL Population Initialization

The DOL-based initialization process is defined as follows:

$$P_{ij}^{DO} = P_{ij} + r1_i \cdot (r2_i \cdot (a_j + b_j - P_{ij}) - P_{ij}) \quad (10)$$

Let P_{ij} represent the initially generated population through random sampling, and P_{ij}^{DO} denote the corresponding individuals initialized using the dynamic-opposite learning (DOL) mechanism. Here, N is the total number of individuals in the population, i refers to the i^{th} candidate, and j indicates the j^{th} dimension of a solution vector. The parameters $r1$ and $r2$ are independently sampled from a uniform distribution over the interval $[0, 1]$. The weighting factor r is set to 1 during the initialization phase.

Afterward, a total of N individuals are chosen from the combined population pool consisting of P and P^{DO} .

3.2 DOL-Based Generation Jumping

During the optimization iterations, the DOL mechanism is periodically activated to enhance population diversity and help the algorithm escape from local optima. This process, referred to as generation jumping, is controlled by a predefined jumping rate, denoted as Jr . In each generation, a random number $rand$ is generated from a uniform distribution in $[0, 1]$. If $rand < Jr$, the DOL-based generation jumping mechanism is triggered for the entire population. The candidate update process then follows the equation below:

$$P_{ij}^{DO} = P_{ij} + w \cdot r3_i \cdot (r4_i \cdot (a_j + b_j - P_{ij}) - P_{ij}) \quad (11)$$

To ensure the updated individuals remain confined within a contracted search region, the interval bounds are adaptively recalculated using:

$$a_j = \min(P_{ij}), \quad b_j = \max(P_{ij}) \quad (12)$$

To this end, a new variant of MFO, namely DOLDEMFO, has been developed. To clearly present the specific algorithmic process⁶⁸, its step-by-step implementation is outlined in Algorithm1. The workflow of DOLDEMFO is illustrated in Fig.1.

Table 1. Parameter settings for optimization algorithms and benchmark problems

Parameters	Value
Population size	100
Max generations for FJSP	200
Max generations for benchmark functions	3000
Flame reduction coefficient J_r	1
Independent runs per experiment	30
Logarithmic spiral constant b (MFO/-DOLDEMFO)	1
Search coefficient t (MFO/DOLDEMFO)	$(-1 - \frac{G}{G_m} - 1) \cdot rand + 1$
Differential Evolution scale factor F	0.7
Differential Evolution crossover rate CR	0.9

4 Experimental results and discussions

In this section, numerical results of the proposed method on benchmark tests and FJSP applications are well demonstrated and discussed. The advantage of the proposed method is comprehensively illustrated.

4.1 Benchmark test

4.1.1 Benchmark test functions

To rigorously evaluate the performance of the proposed DOLDEMFO algorithm, 22 benchmark functions from the CEC 2014 test suite are employed. These functions are widely used in the evolutionary computation community and are categorized into three types—unimodal, multimodal, and hybrid—each emphasizing different optimization challenges.

Unimodal Functions (F1–F3): This group comprises High Conditioned Elliptic (F1), Bent Cigar (F2), and Discus (F3), with global optima located at 100, 200, and 300, respectively. All functions are set in a 30-dimensional space and primarily assess an algorithm's local search efficiency, as they contain only one global optimum and no local optima.

Multimodal Functions (F4–F16): Functions in this category, such as Rosenbrock (F4), Ackley (F5), and Rastrigin (F8), introduce numerous local optima into the landscape, making them suitable for evaluating global exploration capabilities. These problems are more complex due to their deceptive search spaces. The optima of these functions range from 400 to 1600, all defined in 30-dimensional domains.

Hybrid Functions (F17–F22): This set includes HF1 through HF6 (F17–F22), with global optima ranging from 1700 to 2200. These functions blend characteristics of several basic functions, forming diverse landscapes with varying modality and separability. They are specifically designed to test both exploration and exploitation under highly dynamic and irregular conditions.

All benchmark problems are configured in accordance with CEC 2014 standards, ensuring consistency in dimensionality, boundary constraints, and objective definitions.

4.1.2 Parameter settings

Numerical experiments are conducted to compare the proposed DOLDEMFO and other well-known algorithms including teaching-learning-based optimization (TLBO)³¹, Jaya⁶⁹, grey wolf optimizer (GWO)⁷⁰ and differential evolution (DE)⁷¹. These algorithms were selected based on an extensive literature review. They represent a diverse range of optimization philosophies, including teaching-based, swarm-intelligence-based, and evolutionary approaches, and are widely recognized as standard benchmarks in the field. Furthermore, the optimization results obtained by the algorithm DOLMFO that means only modifying MFO with DOL is also used in the comparison to show the superiority of DOL. The parameters for these experiments are shown in Table. 1. All of these studies are simulated on an Intel(R) Core(TM) i7-3537U CPU @ 2.50 GHz PC and the Matlab (R) 2019a software platform.

The selection of appropriate parameter values is crucial for the performance of metaheuristic algorithms. For the other compared algorithms, parameters such as population size and maximum generations were set according to the standard practices for the CEC 2014 benchmark suite to ensure a fair comparison with existing studies. The scaling factor F and crossover rate CR for Differential Evolution were set to 0.7 and 0.9 respectively, which are typical and robust values recommended in the original literature.

Table 2. The sensitivity analysis of w

Weight	Mean				Std				Best num
	F1	F3	F4	F7	F1	F3	F4	F7	
$w = 1$	2.94×10^7	8.04×10^3	2.76×10^2	7.80	1.15×10^7	2.38×10^3	5.24×10^1	2.48	2
$w = 2$	4.20×10^7	8.82×10^3	2.57×10^2	7.08	1.79×10^7	3.81×10^3	4.40×10^1	2.86	1
$w = 3$	3.73×10^7	9.36×10^3	2.68×10^2	7.15	1.43×10^7	3.24×10^3	4.85×10^1	1.83	0
$w = 4$	4.06×10^7	8.66×10^3	2.88×10^2	8.37	1.80×10^7	3.62×10^3	6.43×10^1	2.63	0
$w = 5$	3.22×10^7	9.30×10^3	2.45×10^2	8.43	1.38×10^7	5.29×10^3	6.53×10^1	3.13	1
$w = 6$	3.73×10^7	1.10×10^4	2.67×10^2	7.78	1.73×10^7	6.43×10^3	4.82×10^1	2.90	0
$w = 7$	4.26×10^7	1.06×10^4	2.76×10^2	8.07	1.91×10^7	5.03×10^3	4.69×10^1	2.67	0
$w = 8$	3.75×10^7	1.09×10^4	2.61×10^2	8.48	2.06×10^7	4.46×10^3	5.21×10^1	2.51	0
$w = 9$	3.93×10^7	1.01×10^4	2.57×10^2	8.62	1.80×10^7	4.74×10^3	5.10×10^1	3.51	0
$w = 10$	3.71×10^7	1.14×10^4	2.68×10^2	8.22	1.63×10^7	8.76×10^3	5.28×10^1	3.02	0

For our proposed DOLDEMFO, two new key parameters are introduced: w and Jr . To determine their optimal values, an extensive parameter sensitivity analysis^{72,73} was conducted. We selected four representative benchmark functions (F1, F3, F4, and F7) for this analysis. The performance was evaluated based on the mean and standard deviation over 30 independent runs, as well as the number of functions on which a setting achieved the best mean value. As shown in Table 2 and Table 3, the combination of $w=1$ and $Jr=0.1$ yielded the most competitive and stable overall performance. Therefore, this parameter configuration was adopted for all subsequent experiments in this study.

Table 3. The sensitivity analysis of Jr

Jr	Mean				Std				Best num
	F1	F3	F4	F7	F1	F3	F4	F7	
0.1	2.94×10^7	8.04×10^3	2.76×10^2	7.80×10^0	1.15×10^7	2.38×10^3	5.24×10^1	2.48×10^0	4
0.2	4.05×10^7	9.95×10^3	2.90×10^2	1.01×10^1	1.65×10^7	2.76×10^3	5.92×10^1	3.64×10^0	0
0.3	3.94×10^7	1.14×10^4	3.18×10^2	1.03×10^1	1.65×10^7	3.87×10^3	7.08×10^1	3.53×10^0	0
0.4	5.29×10^7	1.52×10^4	3.36×10^2	1.36×10^1	1.80×10^7	4.28×10^3	6.13×10^1	4.71×10^0	0
0.5	5.46×10^7	2.07×10^4	3.87×10^2	1.57×10^1	2.34×10^7	1.16×10^4	8.25×10^1	4.72×10^0	0
0.6	6.72×10^7	2.36×10^4	4.02×10^2	1.84×10^1	2.34×10^7	1.04×10^4	9.30×10^1	5.63×10^0	0
0.7	8.49×10^7	3.68×10^4	4.32×10^2	1.65×10^1	3.30×10^7	1.33×10^4	1.11×10^2	4.73×10^0	0
0.8	1.03×10^8	4.32×10^4	5.49×10^2	2.27×10^1	3.41×10^7	1.12×10^4	1.79×10^2	1.02×10^1	0
0.9	1.04×10^8	4.32×10^4	5.81×10^2	2.42×10^1	3.00×10^7	1.01×10^4	1.58×10^2	8.48×10^0	0
1.0	1.24×10^8	4.54×10^4	6.34×10^2	3.21×10^1	3.03×10^7	8.61×10^3	1.41×10^2	1.14×10^1	0

4.1.3 Analysis of unimodal/multi-modal test functions

The unimodal and multi-modal benchmarks serve to evaluate the exploitation and exploration capabilities of DOLDEMFO, respectively. Statistical indicators, including the mean and standard deviation, for these two categories are presented in Table 4.

As observed from the results, DOLDEMFO consistently outperforms competing algorithms on the unimodal functions $F1-F3$. Compared to the canonical MFO, the integration of DOL and DE enhances the algorithm's ability to explore the search space and accelerates convergence towards the global optimum.

Multi-modal test functions $F4-F15$ are used to validate the exploration capability of DOLDEMFO in many local optima scenarios. Table 4 shows that DOLDEMFO has a better exploration capability than all other algorithms. DOLDEMFO performs best on all multi-modal test functions except $F8$ and $F11$, which is due to the dynamic changing searching space supported by the DOL strategy and the global searching ability of DE.

4.1.4 Analysis of hybrid test functions

Hybrid functions, comprising both unimodal and multi-modal characteristics, are designed to resemble real-world application scenarios. The mean and standard deviation results for all hybrid test functions are presented in Table 5.

It can be observed that DOLDEMFO achieves the best mean performance on all hybrid test functions except $F22$, where DE shows superiority on $F17$, $F20$, and $F21$. These results indicate that DOLDEMFO is comparable to, or even better than, other algorithms, demonstrating strong capabilities in balancing exploration and exploitation under complex optimization conditions.

Table 4. The mean and standard value of unimodal/multi-modal test functions

Algorithms	F1		F2		F3		F4	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DOLDEMFO	5.86×10^4	5.45×10^4	3.38×10^{-12}	0.00×10^0	1.26×10^{-10}	0.00×10^0	1.12×10^{-2}	8.19×10^{-3}
DOLMFO	5.27×10^5	7.96×10^5	1.35×10^3	5.35×10^3	4.77×10^3	2.37×10^3	7.73×10^{-2}	6.81×10^1
MFO	6.46×10^5	2.19×10^6	6.02×10^2	1.83×10^4	2.75×10^3	5.28×10^3	2.12×10^0	7.61×10^1
TLBO	9.03×10^7	6.56×10^7	1.47×10^{10}	9.58×10^9	2.09×10^4	8.36×10^3	1.75×10^3	5.64×10^2
Jaya	5.06×10^6	2.03×10^6	3.29×10^8	1.14×10^8	2.09×10^4	6.11×10^3	1.80×10^2	3.19×10^1
DE	3.95×10^6	2.28×10^6	1.82×10^5	1.08×10^5	1.44×10^0	8.13×10^{-1}	8.86×10^1	3.38×10^0
GWO	4.37×10^7	1.17×10^7	1.61×10^9	9.98×10^8	1.80×10^4	1.73×10^4	2.99×10^2	3.56×10^1
Algorithms	F5		F6		F7		F8	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DOLDEMFO	1.99×10^1	7.17×10^{-4}	2.64×10^0	1.43×10^0	0.00×10^0	0.00×10^0	7.26×10^1	7.03×10^0
DOLMFO	2.00×10^1	3.06×10^{-5}	9.55×10^0	3.20×10^0	1.14×10^{-13}	8.53×10^{-3}	3.78×10^1	4.91×10^0
MFO	2.00×10^1	3.18×10^{-5}	1.41×10^1	3.07×10^0	7.40×10^{-3}	8.52×10^{-3}	4.18×10^1	1.16×10^1
TLBO	2.10×10^1	8.53×10^{-2}	2.51×10^1	3.52×10^0	2.02×10^2	1.05×10^1	1.91×10^2	6.75×10^0
Jaya	2.10×10^1	5.76×10^{-2}	1.94×10^1	3.71×10^0	3.56×10^0	1.13×10^0	1.89×10^2	3.95×10^0
DE	2.10×10^1	5.24×10^{-2}	3.36×10^1	6.42×10^{-1}	1.98×10^{-1}	2.41×10^{-1}	2.12×10^2	3.15×10^0
GWO	2.10×10^1	2.11×10^{-2}	9.29×10^0	1.19×10^0	1.14×10^1	1.19×10^1	7.94×10^1	1.91×10^1
Algorithms	F9		F10		F11		F12	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DOLDEMFO	4.57×10^1	7.03×10^0	1.30×10^2	2.49×10^2	3.00×10^3	6.20×10^2	4.73×10^{-2}	6.38×10^{-3}
DOLMFO	1.04×10^2	3.04×10^0	3.60×10^2	2.36×10^2	2.99×10^3	1.45×10^2	9.03×10^{-2}	1.65×10^{-1}
MFO	1.07×10^1	1.22×10^1	3.68×10^2	4.11×10^2	3.11×10^3	1.26×10^2	6.40×10^{-2}	2.26×10^{-1}
TLBO	1.67×10^2	2.65×10^1	4.59×10^3	3.66×10^2	4.68×10^3	3.89×10^2	7.78×10^{-1}	1.29×10^{-1}
Jaya	1.85×10^2	2.12×10^1	5.02×10^3	2.10×10^2	5.59×10^3	5.95×10^2	2.40×10^0	1.29×10^{-1}
DE	2.18×10^2	9.20×10^0	6.33×10^3	9.44×10^1	6.74×10^3	2.25×10^2	2.16×10^0	1.60×10^{-1}
GWO	4.86×10^1	3.24×10^1	1.47×10^3	7.05×10^2	2.59×10^3	4.17×10^2	2.43×10^{-1}	1.08×10^0
Algorithms	F13		F14		F15		Best num	
	Mean	Std	Mean	Std	Mean	Std		
DOLDEMFO	1.41×10^{-1}	6.28×10^{-2}	1.83×10^{-1}	2.18×10^{-2}	3.06×10^0	2.55×10^0	13	
DOLMFO	3.58×10^{-1}	2.29×10^{-2}	2.40×10^{-1}	6.00×10^{-2}	5.18×10^0	3.36×10^0	1	
MFO	5.35×10^{-1}	1.00×10^{-1}	3.14×10^{-1}	1.93×10^{-2}	1.11×10^1	4.18×10^0	0	
TLBO	3.34×10^0	6.68×10^{-1}	8.43×10^1	2.60×10^1	9.13×10^2	5.86×10^3	0	
Jaya	4.98×10^{-1}	1.11×10^{-1}	2.25×10^{-1}	2.82×10^{-2}	2.22×10^1	2.25×10^0	0	
DE	5.51×10^{-1}	2.12×10^{-2}	2.87×10^{-1}	3.65×10^{-2}	1.81×10^1	1.20×10^0	0	
GWO	3.98×10^{-1}	3.62×10^{-2}	2.20×10^{-1}	8.50×10^0	7.43×10^0	9.79×10^0	1	

4.1.5 Statistical Test Outcomes

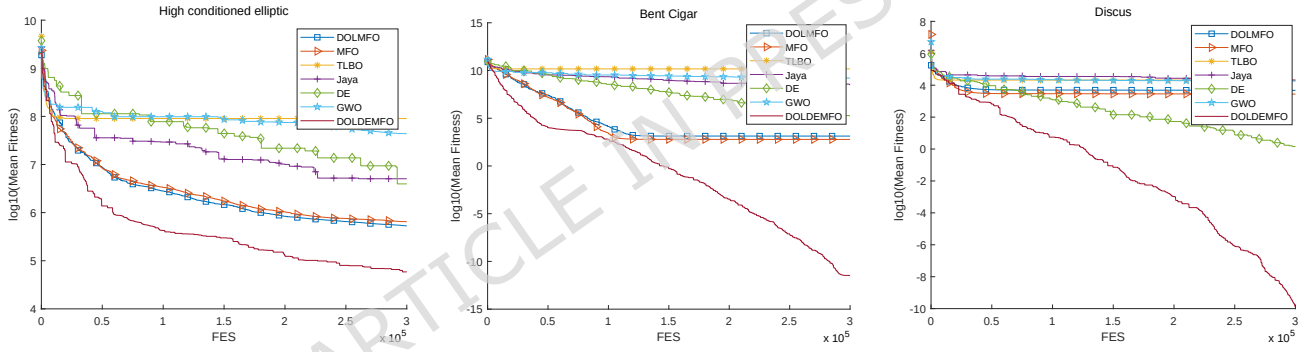
To assess whether there exists a statistically meaningful distinction between the mean values of two datasets, this study applies the Two-Independent-Samples t -test. The obtained t -statistics are presented in Table 6, while their associated P -values are listed in Table 7. A significance level of 0.05 is adopted, which corresponds to the critical value $t(30)_{0.05} = 2.0423$. When the calculated t is less than 2.0423, it implies $P > 0.05$, indicating that the difference between the two compared algorithms is not statistically significant. Conversely, $t > 2.0423$ suggests $P < 0.05$, leading to the rejection of the null hypothesis.

In Table 7, the symbol ‘+’ denotes cases where $P > 0.05$ (i.e., no significant difference), while ‘−’ indicates instances where $P < 0.05$ (i.e., a significant difference exists). The ‘Same’ column counts how often DOLDEMFO performs comparably

Table 5. The mean and standard value of hybrid test functions

Algorithms	F16		F17		F18		F19	
	Mean	Std	Mean	Std	Mean	Std	Mean	Std
DOLDEMFO	1.02×10^1	1.45×10^{-1}	2.03×10^3	3.82×10^1	1.75×10^1	7.13×10^1	2.81×10^0	4.89×10^{-1}
DOLMFO	1.04×10^1	6.73×10^{-1}	1.96×10^4	1.99×10^5	4.50×10^1	1.47×10^3	5.50×10^0	3.03×10^0
MFO	1.04×10^1	1.23×10^0	5.52×10^4	2.30×10^3	8.49×10^2	1.94×10^3	5.56×10^0	6.96×10^{-1}
TLBO	1.10×10^1	4.11×10^{-1}	1.36×10^5	5.55×10^5	4.43×10^2	1.19×10^3	3.22×10^1	2.97×10^1
Jaya	1.25×10^1	1.09×10^{-1}	9.13×10^4	2.17×10^5	6.75×10^5	1.61×10^5	1.03×10^1	1.81×10^0
DE	1.28×10^1	1.53×10^{-1}	2.18×10^3	1.09×10^2	8.67×10^1	4.45×10^0	5.56×10^0	1.04×10^0
GWO	1.09×10^1	5.63×10^{-1}	2.03×10^5	4.60×10^5	5.18×10^3	4.20×10^6	1.90×10^1	6.61×10^1

Algorithms	F20		F21		F22		Best num
	Mean	Std	Mean	Std	Mean	Std	
DOLDEMFO	3.66×10^1	2.31×10^1	7.31×10^2	2.10×10^2	3.81×10^2	5.43×10^1	6
DOLMFO	6.95×10^3	1.51×10^4	1.32×10^4	9.14×10^4	1.60×10^2	1.47×10^2	1
MFO	3.49×10^3	2.18×10^4	1.29×10^5	4.02×10^4	4.13×10^2	1.82×10^2	0
TLBO	4.80×10^3	4.47×10^3	1.95×10^4	3.38×10^4	3.45×10^2	2.50×10^2	0
Jaya	1.61×10^3	2.29×10^2	2.26×10^4	2.66×10^4	5.54×10^2	3.67×10^1	0
DE	5.44×10^1	6.82×10^0	1.34×10^3	4.02×10^1	1.77×10^2	1.31×10^2	0
GWO	4.14×10^3	9.66×10^3	2.17×10^4	2.42×10^5	3.13×10^2	1.14×10^2	0

**Figure 2.** The convergence trends of all algorithms on unimodal functions

to competing approaches, whereas the ‘Better’ column reflects how frequently it significantly outperforms others.

Based on the comparative results summarized in Table 6 and Table 7, it is evident that DOLDEMFO delivers statistically validated enhancements across a majority of test cases involving unimodal, multimodal, and composite benchmark problems. These findings underscore the advantage of embedding DOL and DE techniques into the original MFO framework.

4.1.6 Analysis of convergence

The convergence behaviors of all compared algorithms on unimodal and multi-modal benchmark functions are illustrated in Fig. 2 and Fig. 3, respectively. Meanwhile, the convergence trends for hybrid functions are depicted in Fig. 4.

As observed from Fig. 2–3, DOLDEMFO exhibits sharp fluctuations during the early stages of iteration across most test functions. This phenomenon indicates that DOLDEMFO possesses strong exploration ability, primarily attributed to the dynamic adjustment of the search space by DOL and DE strategies. Over time, the variations in DOLDEMFO gradually diminish, leading the algorithm to steadily converge toward the global optimum. This convergence behavior is largely due to the enhanced exploitation capability provided by the dynamically evolving DOL mechanism. In contrast, the convergence curves of many competing algorithms, such as the standard MFO or GWO, tend to plateau much earlier, suggesting they may have prematurely converged to local optima. The ability of DOLDEMFO to sustain its search for better solutions and ultimately converge to a superior result highlights its enhanced performance in balancing global exploration and local exploitation, leading to stronger robustness and higher solution accuracy across diverse and complex problems.

Table 6. The t values of DOLDEMFO and other algorithms

	DOLDEMFO							
	F1	F2	F3	F4	F5	F6	F7	F8
DOLMFO	2.78×10^3	1.01×10^0	5.36×10^2	4.38×10^{-2}	2.00×10^1	1.75×10^1	0.00×10^0	8.04×10^1
MFO	2.14×10^3	2.43×10^{-1}	2.07×10^2	1.32×10^0	2.00×10^1	2.95×10^1	4.39×10^{-1}	4.80×10^1
TLBO	6.10×10^4	8.22×10^3	1.25×10^3	4.03×10^2	2.05×10^1	5.52×10^1	3.41×10^2	2.37×10^2
Jaya	1.89×10^4	1.68×10^3	1.46×10^3	1.74×10^2	2.49×10^1	4.04×10^1	1.83×10^1	2.95×10^2
DE	1.39×10^4	3.03×10^1	8.74×10^0	2.63×10^2	2.61×10^1	1.17×10^2	2.20×10^0	3.88×10^2
GWO	6.97×10^4	2.79×10^3	7.49×10^2	2.74×10^2	4.07×10^1	2.25×10^1	1.81×10^1	8.36×10^0
	DOLDEMFO							
	F9	F10	F11	F12	F13	F14	F15	F16
DOLMFO	1.00×10^2	5.72×10^1	1.98×10^0	5.68×10^{-1}	4.06×10^0	1.09×10^0	4.77×10^0	1.21×10^0
MFO	4.37×10^1	5.07×10^1	2.20×10^1	1.89×10^{-1}	5.34×10^0	3.53×10^0	1.69×10^1	9.34×10^{-1}
TLBO	1.14×10^2	9.85×10^1	2.89×10^2	1.08×10^1	2.04×10^1	9.03×10^1	6.50×10^1	5.87×10^0
Jaya	2.95×10^2	1.43×10^2	1.25×10^3	4.06×10^2	3.50×10^1	4.69×10^0	1.02×10^0	2.49×10^1
DE	3.88×10^2	2.34×10^2	1.83×10^3	7.04×10^2	2.83×10^1	7.74×10^0	2.35×10^0	2.60×10^1
GWO	2.52×10^0	2.37×10^2	6.97×10^1	1.02×10^0	4.47×10^0	6.94×10^{-2}	6.81×10^0	4.55×10^0
	DOLDEMFO							
	F17	F18	F19	F20	F21	F22		
DOLMFO	2.15×10^2	3.83×10^0	7.85×10^0	3.07×10^2	2.25×10^2	8.53×10^1		
MFO	6.02×10^3	1.01×10^2	1.38×10^1	1.28×10^2	3.49×10^3	1.14×10^1		
TLBO	9.84×10^2	6.56×10^1	2.92×10^1	3.89×10^2	5.57×10^2	1.13×10^1		
Jaya	1.04×10^3	9.21×10^3	2.70×10^1	5.42×10^2	7.31×10^2	9.93×10^1		
DE	6.77×10^1	4.35×10^1	1.21×10^1	1.78×10^1	2.10×10^2	8.20×10^1		
GWO	1.62×10^3	1.37×10^1	1.08×10^1	2.28×10^2	2.33×10^2	2.87×10^1		

Table 7. The P values of DOLDEMFO and other algorithms

	DOLDEMFO											
	F1	F2	F3	F4	F5	F6	F7	F8	F9	F10	F11	F12
DOLMFO	–	+	–	+	–	–	+	–	–	–	+	+
MFO	–	+	–	+	–	–	+	–	–	–	–	+
TLBO	–	–	–	–	–	–	–	–	–	–	–	–
Jaya	–	–	–	–	–	–	–	–	–	–	–	–
DE	–	–	–	–	–	–	–	–	–	–	–	–
GWO	–	–	–	–	–	–	–	–	–	–	–	+
	DOLDEMFO										Same	Better
	F13	F14	F15	F16	F17	F18	F19	F20	F21	F22		
DOLMFO	–	+	–	+	–	–	–	–	–	–	7	15
MFO	–	–	–	+	–	–	–	–	–	–	5	17
TLBO	–	–	–	–	–	–	–	–	–	–	0	22
Jaya	–	–	+	–	–	–	–	–	–	–	1	21
DE	–	–	–	–	–	–	–	–	–	–	0	22
GWO	–	+	–	–	–	–	–	–	–	–	2	20

5 Industrial Optimization Problem Applications

5.1 Optimal Trajectory Planning Problem

In most cases, trajectory planning is executed within the joint space of robotic systems. Although planning in the operational space offers enhanced control over the end-effector's motion, joint space planning is typically favored due to its ability to

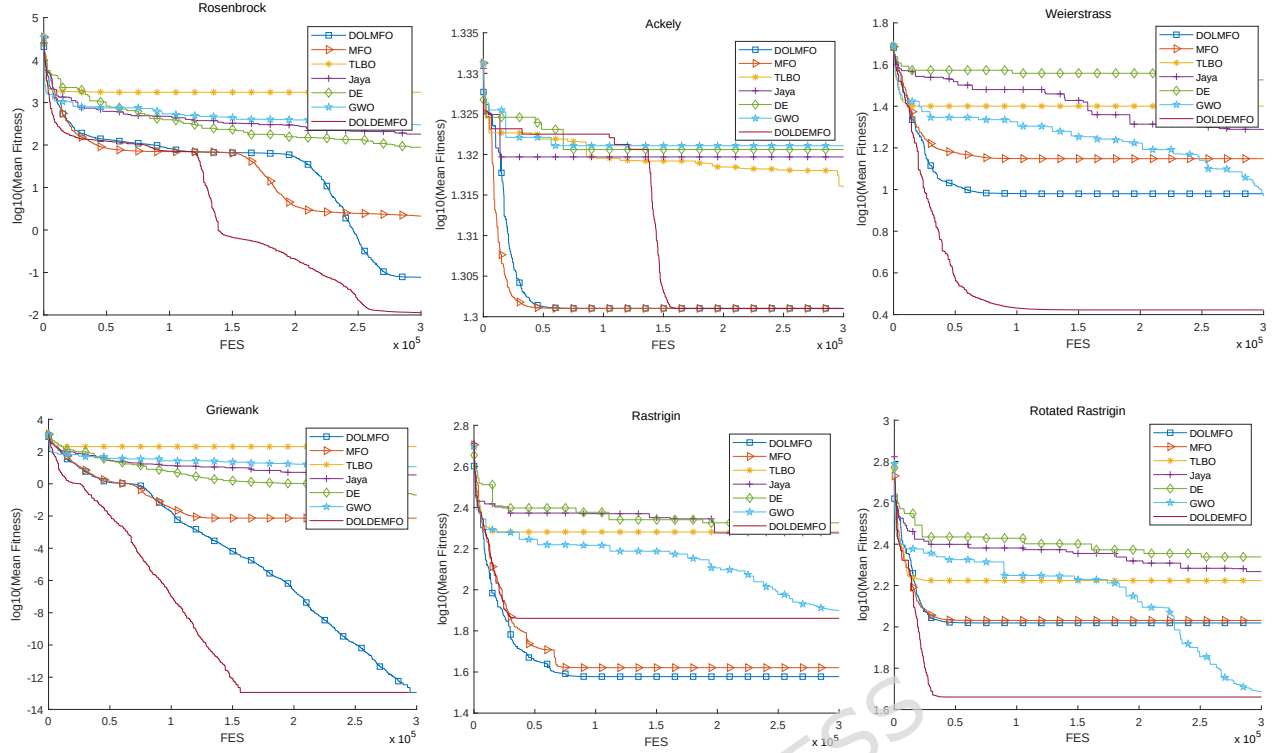


Figure 3. The convergence trends of all algorithms on multi-modal functions (1/2)

suppress abrupt joint transitions and ensure the smoothness of manipulator motion. Hence, this study focuses on trajectory design in the joint domain.

The planning task is modeled as a multi-objective optimization problem, where the goal is to simultaneously minimize: (1) the integral of the squared joint acceleration over the entire trajectory, and (2) the total time required for execution. These two criteria inherently conflict with one another. Specifically, minimizing execution time tends to increase dynamic load, whereas minimizing acceleration smooths the path but may extend execution time.

A compromise between these two objectives is achieved by tuning their respective weight coefficients, as suggested in¹⁶. The resulting optimization formulation is described as follows:

$$\text{find: } \min_{w_T} N \sum_{i=1}^{N_p-1} T_i + w_J \sum_{j=1}^N \int_0^{T_f} (J_j(t))^2 dt \quad (13)$$

$$\text{subject to: } P_j^{\text{lower}} \leq P_j(t) \leq P_j^{\text{upper}}, \quad j = 1, \dots, N, \quad (14)$$

$$|V_j(t)| \leq VL_j, \quad j = 1, \dots, N, \quad (15)$$

$$|A_j(t)| \leq AL_j, \quad j = 1, \dots, N, \quad (16)$$

$$|J_j(t)| \leq JL_j, \quad j = 1, \dots, N. \quad (17)$$

In this optimization context, multiple B-spline curves are essential, as they offer the necessary flexibility and local support required to meet the desired trajectory characteristics. The formulation of a B-spline curve is given by:

$$p(u) = \sum_{i=0}^n d_i N_{i,k}(u) \quad (18)$$

where $p(u)$ represents the joint position, d_i denotes the i th control point of the B-spline, k is the degree of the B-spline, and $N_{i,k}(u)$ refers to the i th basis function associated with the k th order B-spline. Here, n is the total number of control points, and u

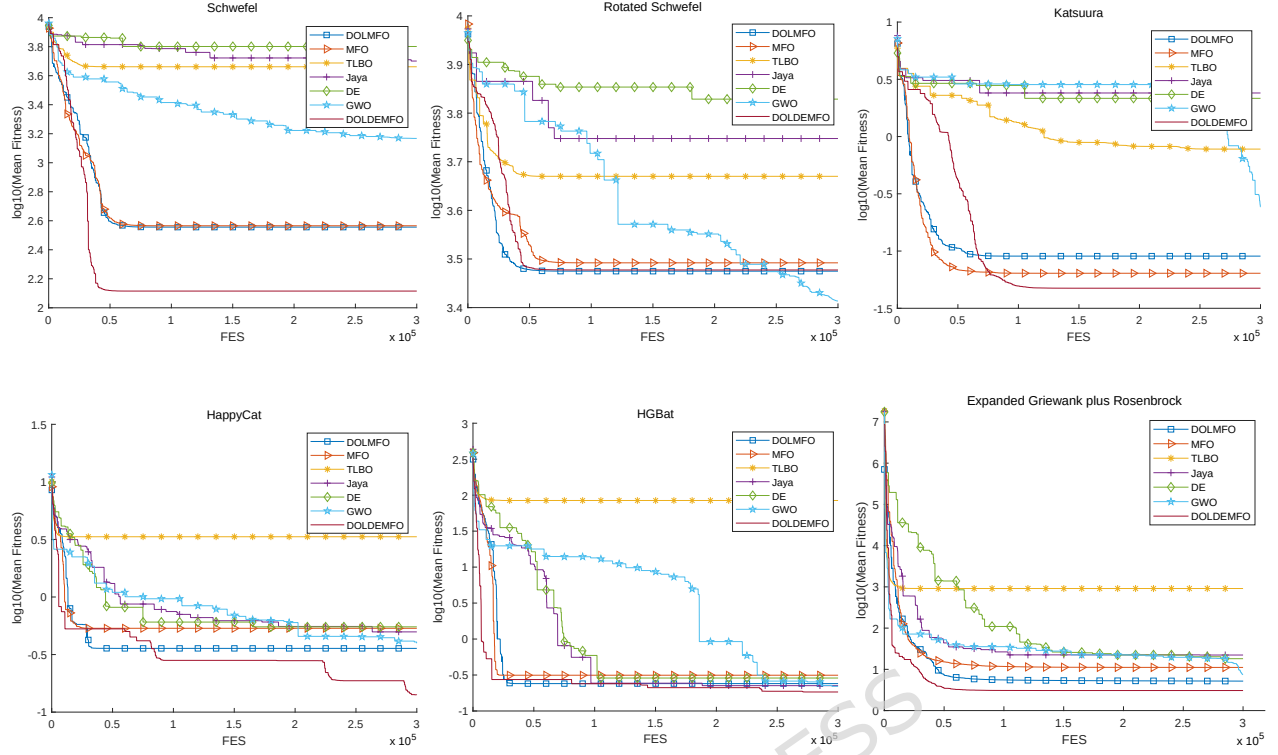


Figure 3. The convergence trends of all algorithms on multi-modal functions (2/2)

Table 8. Symbol definitions for trajectory optimization.

Symbol	Meaning
N	Total number of robot joints
w_T	Weight factor associated with execution time
N_p	Number of via-points
T_i	Time interval between adjacent via-points
$P_j(t)$	Position of joint j at time t
$V_j(t)$	Velocity of joint j at time t
$A_j(t)$	Acceleration of joint j at time t
$J_j(t)$	Jerk of joint j at time t
w_J	Weight factor related to jerk
T_f	Total trajectory execution time
P_j^{upper}	Upper limit of joint j position
P_j^{lower}	Lower limit of joint j position
VL_j	Symmetric velocity limit of joint j
AL_j	Symmetric acceleration limit of joint j
JL_j	Symmetric jerk limit of joint j

denotes the normalized knot vector variable. The basis functions $N_{i,k}(u)$ are computed through the Cox–de Boor recursive formula.

The basis functions $N_{i,k}(u)$ are recursively defined as follows:

$$N_{i,0}(u) = \begin{cases} 1, & u_i \leq u < u_{i+1} \\ 0, & \text{otherwise} \end{cases} \quad (19)$$

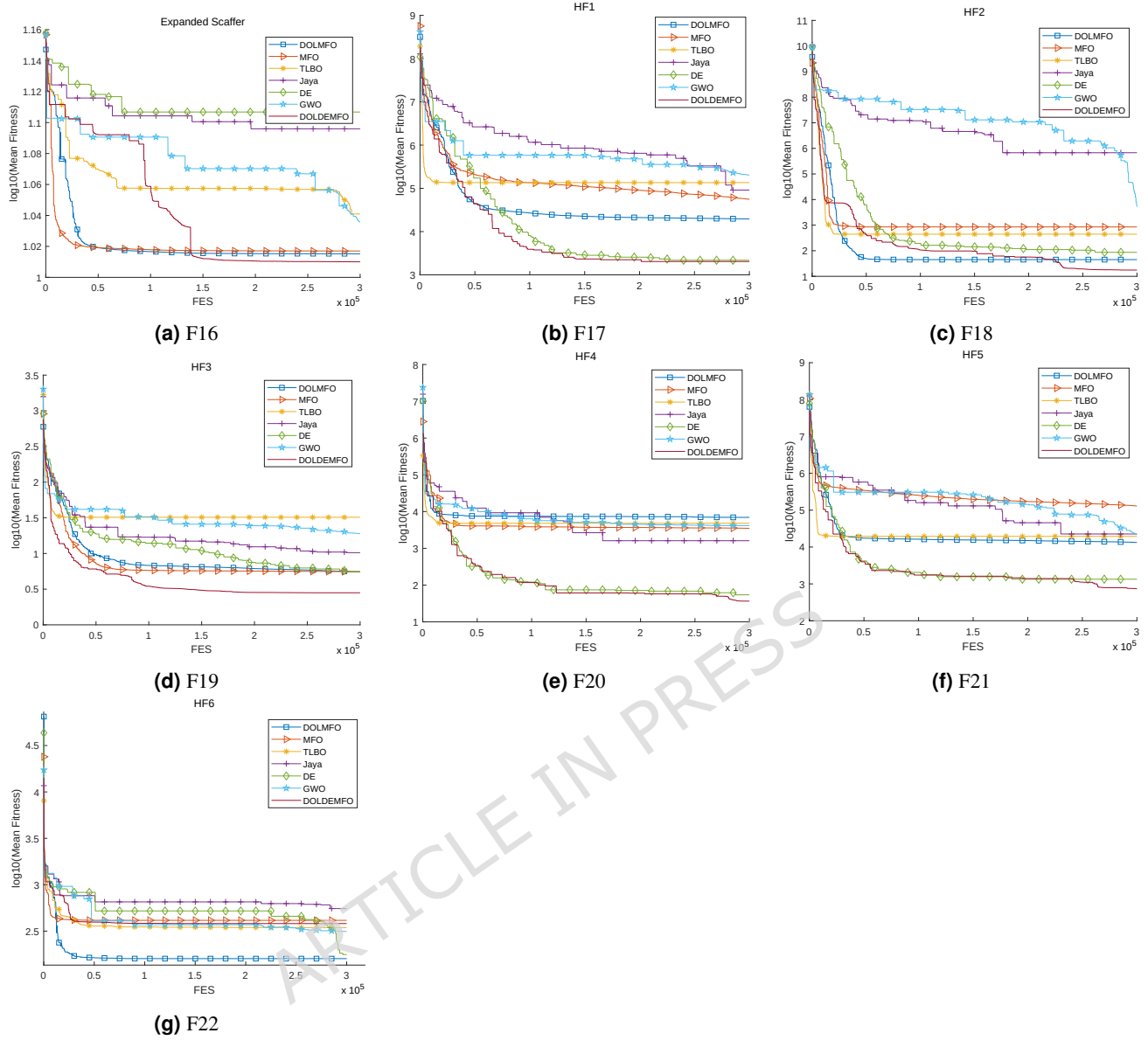


Figure 4. The convergence trends of all algorithms on hybrid functions

$$N_{i,k}(u) = \frac{u - u_i}{u_{i+k} - u_i} N_{i,k-1}(u) + \frac{u_{i+k+1} - u}{u_{i+k+1} - u_{i+1}} N_{i+1,k-1}(u) \quad (20)$$

This recursive formulation ensures that for any parameter u within the interval $[u_i, u_{i+k+1}]$, only $k + 1$ basis functions contribute, endowing the B-spline with its highly desirable local support property.

To map this into the normalized parameter domain, we transform the knot vector $U = [u_0, u_1, \dots, u_{n+2k}]$, ensuring that $u_k = 0$ and $u_{n+k} = 1$. The parameterization then adopts an arc-length approach to preserve the relative distances along the trajectory:

$$u_i = u_{i-1} + \frac{\Delta t_{i-k-1}}{\sum_{r=0}^{n-1} |\Delta t_r|}, \quad i = k + 1, k + 2, \dots, k + n \quad (21)$$

The de Boor-Cox algorithm, a cornerstone of B-spline theory, enables efficient calculation of the derivatives at different levels, which are essential for evaluating the trajectory's velocity, acceleration, and jerk respectively:

$$p^{(l)}(u) = \sum_{r=i-k+l}^i d_r^{(l)} N_{r,k-l}(u), \quad u_i \leq u < u_{i+1} \quad (22)$$

where

$$d_r^{(l)} = (k+1-l) \left(\frac{d_r^{(l-1)} - d_{r-1}^{(l-1)}}{u_{r+k+1-l} - u_r} \right) \quad (23)$$

The core of trajectory optimization lies in ensuring that the B-spline curve passes through the specified points Q , while also satisfying the desired velocity and acceleration constraints. This requirement leads to a system of linear equations that define the control points d_r :

$$p(u_{i+k}) = \sum_{r=i}^{i+k} d_r N_{r,k}(u) = p_i, \quad i = 0, 1, 2, \dots, n \quad (24)$$

In addition, boundary conditions are incorporated to manage the start and end states. For a fifth-degree B-spline ($k = 5$), these conditions are expressed as:

$$\begin{cases} p^{(1)}(u_0) = v_0, & p^{(2)}(u_0) = a_0 \\ p^{(1)}(u_e) = v_e, & p^{(2)}(u_e) = a_e \end{cases} \quad (25)$$

These equations ensure that the trajectory meets both the desired initial and terminal velocities (v_0, v_e) and accelerations (a_0, a_e), providing a smooth and efficient path. The complete system is then represented in matrix form:

$$A_j d_j = p_j \quad (26)$$

where A_j is constructed from the B-spline basis functions, and d_j and p_j are vectors of control points and the specified trajectory points, respectively. Solving this system yields the control points:

$$d_j = A_j^{-1} p_j \quad (27)$$

Finally, the derivative properties of the B-spline curve are derived to yield the velocity, acceleration, and jerk of the trajectory:

$$\begin{cases} v(t) = \dot{p}(u) = \sum_{r=i-k+1}^i d_r^{(1)} N_{r,k-1}(u) \\ a(t) = \ddot{p}(u) = \sum_{r=i-k+2}^i d_r^{(2)} N_{r,k-2}(u) \\ j(t) = \dddot{p}(u) = \sum_{r=i-k+3}^i d_r^{(3)} N_{r,k-3}(u) \end{cases} \quad (28)$$

By incorporating the B-spline formulation into trajectory optimization, we achieve a path that is smooth and precise while rigorously adhering to dynamic constraints^{74,75}. This method effectively balances the dual objectives of minimizing execution time and maintaining kinematic continuity.

To further enhance trajectory planning, the integration of the B-spline approach with the DOLDEMFO algorithm offers a powerful and adaptive solution^{76,77}. This fusion effectively harnesses the strengths of both methods, enabling the generation of trajectories that are not only smooth and efficient but also highly adaptable to complex, dynamic environments. As shown in Figure 5, the DOLDEMFO algorithm precisely optimizes the B-spline control points, allowing for an exceptional balance between smoothness, accuracy, and execution time, even under intricate motion constraints.

The trajectory of industrial robots is discretized into a series of design points along the time scale. To ensure zero jerk at both the beginning and end of the motion, two auxiliary points are inserted at the second and penultimate positions within the design point sequence. The standardized node vector u is subsequently generated based on the time sequence t corresponding to these design points.

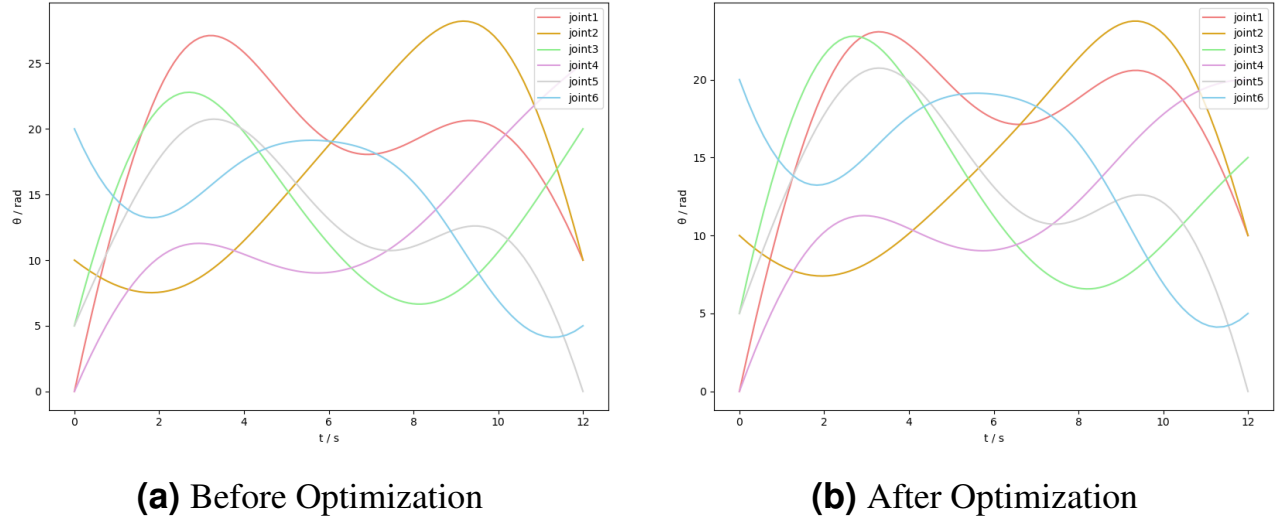


Figure 5. Comparison of Joint Angles Before and After DOLDEMFO Optimization of the B-Spline Generated Trajectory

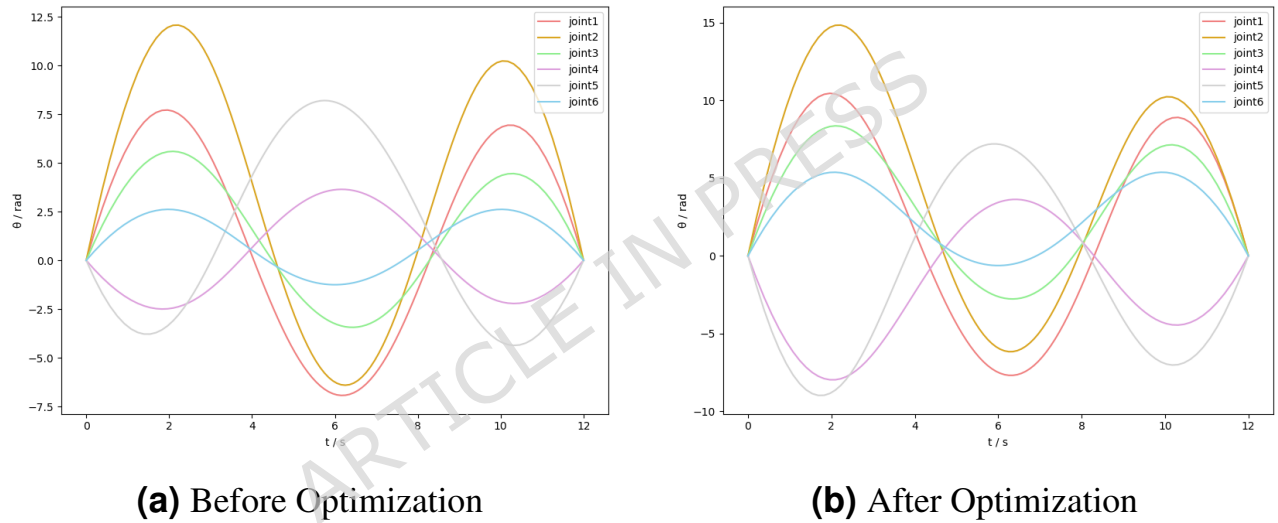


Figure 6. Comparison of Joint Velocities Before and After DOLDEMFO Optimization of the B-Spline Generated Trajectory

5.2 Flexible Job-shop Scheduling Problem

The Flexible Job-shop Scheduling Problem (FJSP) is formulated as follows: A total of N jobs are assigned for processing across M machines within a workshop. Each job comprises a sequence of operations, where each operation must be processed by precisely one machine from a predefined set. The scheduling objective addressed in this study involves minimizing the overall completion time, commonly referred to as the makespan ($C_{\max}$), across all jobs assigned to machines. The formal mathematical formulation is given below, following the definitions in³⁵:

Notations:

- i : Index of the i^{th} job
- j : Index of the j^{th} operation within the i^{th} job
- k : Index of the k^{th} machine
- S_{ij} : Start time of operation j in job i
- S_{ik} : Start time of job i processed on machine k

- S_{ijk} : Start time of operation j in job i executed on machine k
- P_{ij} : Processing time of the j^{th} operation of job i
- P_{ik} : Processing duration of job i on machine k
- P_{ijk} : Processing duration of operation j of job i on machine k
- T_i : Completion time of job i
- T_{ijk} : Completion time of operation j of job i on machine k

Optimization Goal: $\min C_{\max}$

Constraints:

- Jobs are executed independently, and no interruption or cancellation is permitted once processing begins.

$$S_{ijk} + P_{ijk} = T_{ijk} \quad (29)$$

$$\sum_k S_{ijk} \geq \sum_k T_{i(j-1)k} \quad \forall i, j > 1 \quad (30)$$

- Each machine is capable of handling only one job operation at any given time.

$$S_{ijk} + P_{ijk} \leq S_{(i+1)jk} \quad \forall j \quad (31)$$

- Each operation within a job is assigned to only one machine and is processed without interruption.

$$S_{ijk} + P_{ijk} \leq S_{i(j+1)k} \quad \forall k \quad (32)$$

- Machines and jobs are assumed to be available from time zero; transmission delays are considered negligible.

$$S_{ijk} \geq 0 \quad (33)$$

$$T_{ijk} \geq 0 \quad (34)$$

- Processing times are known in advance and include set-up, material transfer, and other overhead costs.

$$T_i \geq T_{ij} \quad \forall j \quad (35)$$

$$C_{\max} \geq T_i \quad \forall i \quad (36)$$

The Flexible Job Shop Scheduling Problem (FJSP) is recognized as a representative combinatorial optimization problem that falls under the class of NP-hard problems. Its complexity arises from the combination of discrete machine assignment and operation sequencing decisions, coupled with strong inter-operation dependencies and numerous constraints. Efficiently solving such problems requires advanced metaheuristics capable of navigating large and highly constrained search spaces.

5.2.1 Encoding and Decoding

In each iteration of the FJSP solution process, three core variables are employed to represent candidate solutions: X , Y_1 , and Y_2 . The matrix X designates the selected machine for each operation associated with all jobs. Variable Y_1 denotes the corresponding operation start times, while Y_2 records their completion times. The encoding scheme is outlined below:

- The solution is structured using matrices of dimension $m \times n$, where m is the total number of jobs, and n represents the number of operations per job.
- In this formulation, X determines the machine assignment, Y_1 holds the start time information, and Y_2 stores the respective completion times for all operations.
- The initial values of X are assigned by randomly selecting machines from the feasible machine alternatives for each operation. Based on this assignment, both Y_1 and Y_2 are derived accordingly.
- During the optimization phase utilizing DOLDEMFO, each row of X is interpreted as an independent decision vector corresponding to a job.

The decoding procedure proceeds as follows:

1. **Step 1:** Identify the individual X with the highest fitness value.
2. **Step 2:** Construct Y_1 by enforcing all scheduling constraints. In particular, the start time of each operation must not precede the completion time of its immediate predecessor on the same machine. If the final operation of a job finishes later than that of a prior job on the same resource, its successor's start time is adjusted accordingly.
3. **Step 3:** Calculate Y_2 for every operation as the sum of its start time and corresponding processing time.
4. **Step 4:** Use Y_1 and Y_2 to build a Gantt chart, where the i^{th} appearance of a numerical identifier within each row indicates the i^{th} operation of the respective job.

To clearly illustrate the decoding process, an example is provided in Fig. 7.

5.2.2 Application to FJSP

Beyond the evaluation on numerical benchmark functions, the proposed DOLDEMFO is further applied to solve the Flexible Job-shop Scheduling Problem (FJSP) to comprehensively assess its performance in a practical, complex optimization scenario. The primary objective for the selected FJSP instances is the minimization of the makespan, a standard metric for scheduling efficiency^{78–82}.

To illustrate the algorithm's effectiveness, we present a detailed analysis of its performance on several representative benchmark instances from the literature. The convergence behavior of DOLDEMFO is showcased on two medium-to-large-scale problems, MFJS02 and MFJS06. Fig. 8 presents the convergence curve for the makespan on the MFJS02 problem, while Fig. 9 displays the convergence on MFJS06. Both figures consistently reveal that DOLDEMFO exhibits rapid convergence characteristics. In the early stages of the search, the algorithm quickly finds high-quality solutions, and it continues to refine these solutions steadily until it converges to the optimal or a near-optimal makespan. This demonstrates the algorithm's strong ability to effectively navigate the complex combinatorial search space of the FJSP.

Furthermore, to visualize the quality of the final solutions, the optimal schedules obtained by DOLDEMFO for a small-scale problem (MFJS01) and a large-scale problem (MFJS10) are depicted as Gantt charts. Fig. 10 shows the Gantt chart for MFJS01, and Fig. 11 shows the chart for MFJS10. These charts illustrate the optimized allocation of operations to machines over time. The compactness of the schedules, particularly for the large-scale MFJS10 instance, visually confirms the algorithm's capability to find highly efficient solutions that minimize the total completion time.

In summary, the analysis of both the convergence dynamics and the final scheduling solutions on these representative FJSP instances validates the applicability and effectiveness of the proposed DOLDEMFO algorithm for solving complex industrial optimization problems.

6 Conclusions

In this paper, The learning strategy named DOL and DE are embedded into MFO to prevent it from premature convergence, thus generating a novel DOLDEMFO algorithm. The asymmetric and dynamic properties of DOL helps DOLDEMFO keep effective exploration and exploitation capabilities in dealing with complex optimization problems. Numerical comparison between the proposed method and several counterparts has been conducted on 20 benchmarks from CEC 2014 and 20 FJSP problems to

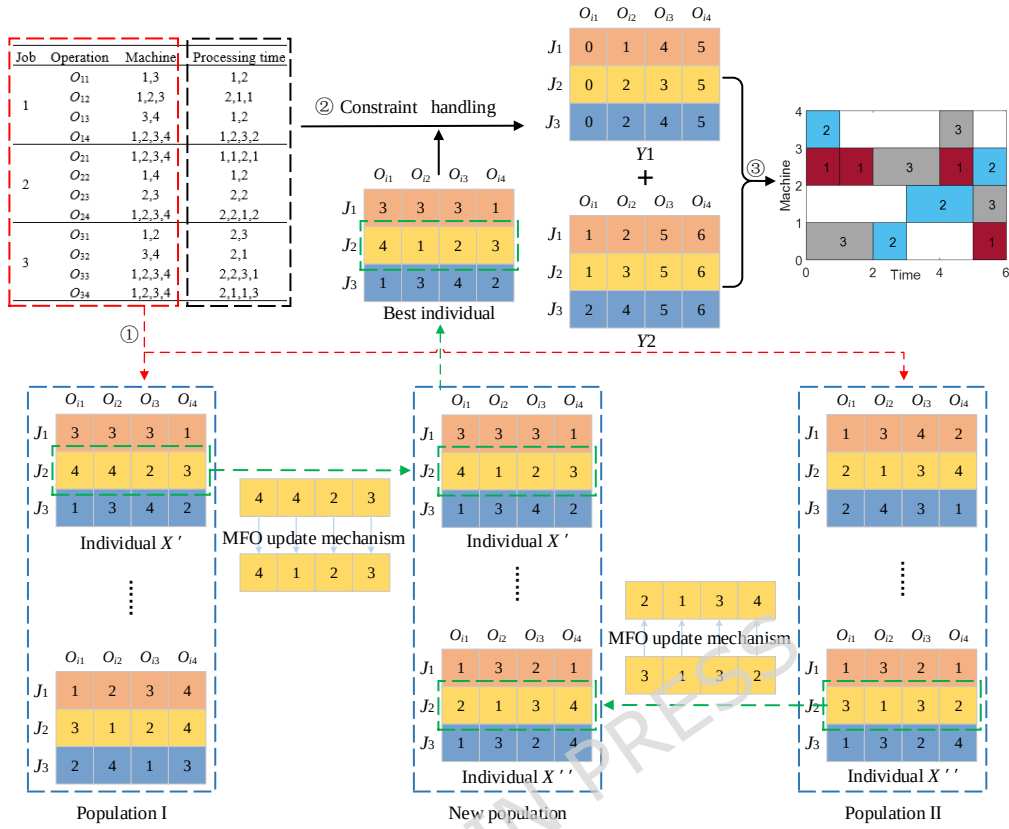


Figure 7. The encoding and decoding process

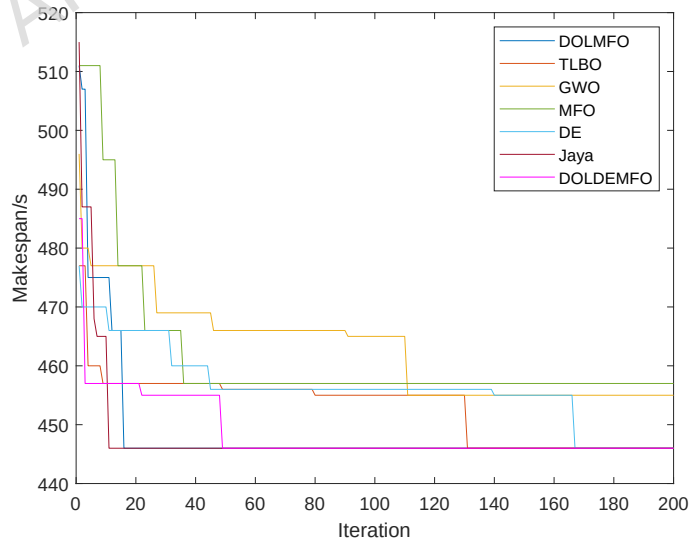


Figure 8. Convergence results of MFJS02 for all the compared algorithms

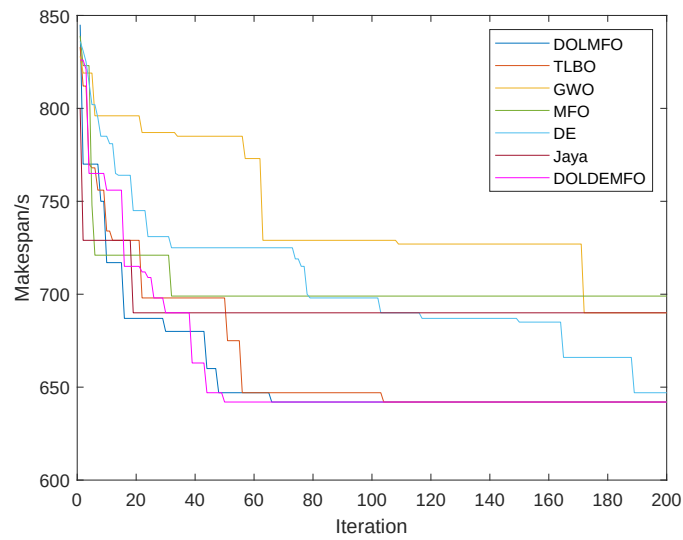


Figure 9. Convergence results of MFJS02 for all the compared algorithms

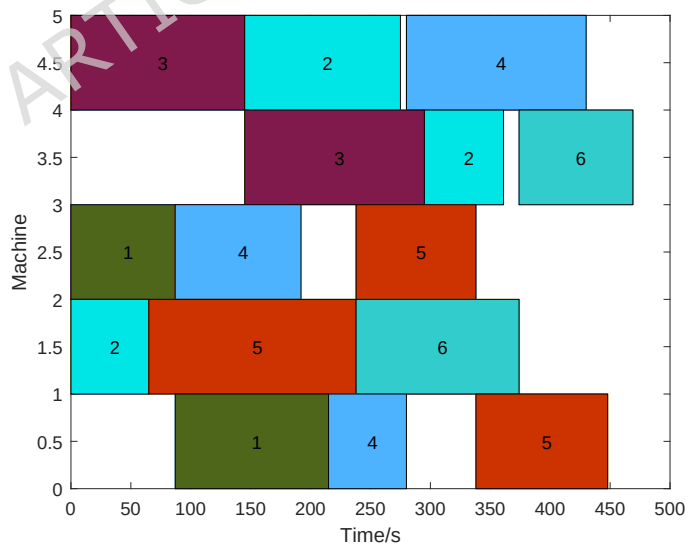


Figure 10. Gantt chart of an optimal solution to MFJS01 obtained by DOLDEMFO

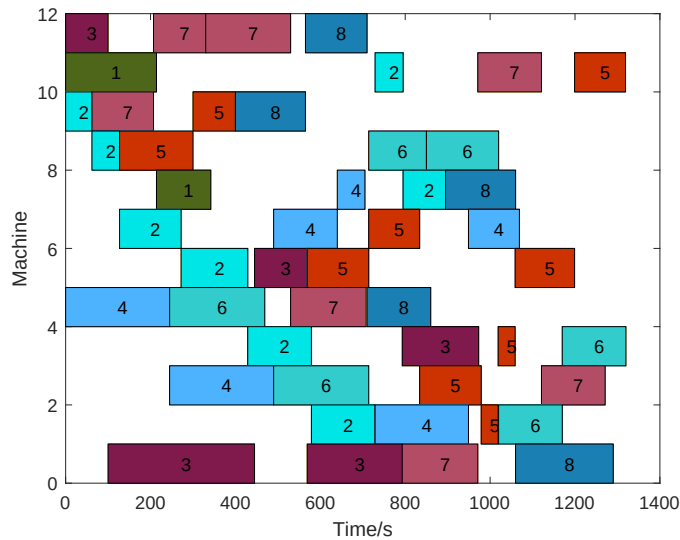


Figure 11. Gantt chart of an optimal solution to MFJS10 obtained by DOLDEMFO

evaluate the performance of DOLDEMFO. Comprehensive results show that the proposed DOLDEMFO, compared to other popular methods, achieves a highly competitive performance in solving both numerical benchmarks and complex industrial optimization problem including trajectory optimization and FJSP problems in different scales. It is worth recommending as a promising tool for science and engineering optimization problems in practice. Admittedly, the proposed method has certain limitations, as its performance may not be optimal in some specific scenarios; nevertheless, the overall results confirm that it demonstrates a strong performance in the vast majority of the tested cases. Future work will be dedicated to adapting this method for application in more complex optimization environments.

Funding

This research work was supported by the National Natural Science Foundation of China under grants 52077213 and 62003332, and by the Youth Innovation Promotion Association CAS (Grant No. 2021358).

Author contributions statement

Conceptualization, Hao Wu and Zhile Yang; methodology, Lijuan Li; software, Hao Wu; validation, Guohui Wang, Y.G., and Binhong Shi; formal analysis, Hao Wu and Zhile Yang; investigation, Lijuan Li and Zhile Yang; resources, Lijuan Li and Zhile Yang; data curation, Lijuan Li and Zhile Yang; writing—original draft preparation, Linxin Zhang; writing—review and editing, Linxin Zhang; visualization, Linxin Zhang; assistance with preliminary tasks, Binhong Shi; supervision, Lijuan Li and Zhile Yang; project administration, Lijuan Li and Guohui Wang; funding acquisition, Lijuan Li and Zhile Yang. All authors have read and agreed to the published version of the manuscript.

Competing interests

The authors declare no competing interests.

Data availability

The benchmark datasets used in this study are publicly available in the evolutionary computation literature. The industrial datasets used in the application case studies involve confidential commercial information and are not publicly available. However, these data may be available from the corresponding author upon reasonable request and with permission from the relevant industrial partners.

References

1. Mirjalili, S. Moth-flame optimization algorithm: A novel nature-inspired heuristic paradigm. *Knowledge-Based Syst.* **89**, 228–249 (2015).

2. Li, C., Li, S. & Liu, Y. A least squares support vector machine model optimized by moth-flame optimization algorithm for annual power load forecasting. *Appl. Intell.* **45**, 1166–1178 (2016).
3. Savsani, V. & Tawhid, M. A. Non-dominated sorting moth flame optimization (ns-mfo) for multi-objective problems. *Eng. Appl. Artif. Intell.* **63**, 20–32 (2017).
4. Bahrami, M., Bozorghaddad, O. & Chu, X. Moth-flame optimization (mfo) algorithm. *Bozorg-Haddad O.(eds) Adv. Optim. by Nature-Inspired Algorithms. Stud. Comput. Intell.* **720**, 131–141 (2018).
5. Li, C. *et al.* A double evolutionary learning moth-flame optimization for real-parameter global optimization problems. *IEEE Access* **6**, 76700–76727 (2018).
6. Buch, H., Trivedi, I. N. & Jangir, P. Moth flame optimization to solve optimal power flow with non-parametric statistical evaluation validation. *Cogent Eng.* **4**, 1286731 (2017).
7. Deb, K. Constrained multi-objective evolutionary algorithm. *Evol. Swarm Intell. Algorithms* 85–118 (2019).
8. Yildiz, B. S. & Yildiz, A. R. Moth-flame optimization algorithm to determine optimal machining parameters in manufacturing processes. *Mater. Testing-Materials Components Technol. Appl.* **59**, 425–429 (2017).
9. Jia, H., Ma, J. & Song, W. Multilevel thresholding segmentation for color image using modified moth-flame optimization. *IEEE Access* **7**, 44097–44134 (2019).
10. Luo, J. *et al.* A new kernel extreme learning machine framework for somatization disorder diagnosis. *IEEE Access* **7**, 45512–45525 (2019).
11. Xu, Y. *et al.* A new hybrid machine learning approach for prediction of phenanthrene toxicity on mice. *IEEE Access* 1–1 (2019).
12. Si, L., Wang, Z. & Jiang, G. Fusion recognition of shearer coal-rock cutting state based on improved rbf neural network and d-s evidence theory. *IEEE Access* **7**, 122106–122121 (2019).
13. Ji, B., Yuan, X. & Yuan, Y. A hybrid intelligent approach for co-scheduling of cascaded locks with multiple chambers. *IEEE Transactions on Syst. Man, Cybern.* **49**, 1236–1248 (2019).
14. Sapre, S. & Mini, S. Opposition-based moth flame optimization with cauchy mutation and evolutionary boundary constraint handling for global optimization. *soft computing* **23**, 6023–6041 (2019).
15. Gasparetto, A., Boscariol, P., Lanzutti, A. & Vidoni, R. Path planning and trajectory planning algorithms: A general overview. *Motion operation planning robotic systems: Backgr. practical approaches* 3–27 (2015).
16. Huang, J., Hu, P., Wu, K. & Zeng, M. Optimal time-jerk trajectory planning for industrial robots. *Mech. Mach. Theory* **121**, 530–544 (2018).
17. Soori, M., Arezoo, B. & Dastres, R. Optimization of energy consumption in industrial robots, a review. *Cogn. Robotics* **3**, 142–157 (2023).
18. Xu, Z., Wang, W., Chi, Y., Li, K. & He, L. Optimal trajectory planning for manipulators with efficiency and smoothness constraint. *Electronics* **12**, 2928 (2023).
19. Zhang, X. & Shi, G. Multi-objective optimal trajectory planning for manipulators in the presence of obstacles. *Robotica* **40**, 888–906 (2022).
20. Wang, F., Wu, Z. & Bao, T. Time-jerk optimal trajectory planning of industrial robots based on a hybrid woa-ga algorithm. *Processes* **10**, 1014 (2022).
21. Zhao, J., Zhu, X. & Song, T. Serial manipulator time-jerk optimal trajectory planning based on hybrid iwoa-pso algorithm. *IEEE Access* **10**, 6592–6604 (2022).
22. Zhang, L., Hu, T., Yang, Z., Yang, D. & Zhang, J. Elite and dynamic opposite learning enhanced sine cosine algorithm for application to plat-fin heat exchangers design problem. *Neural Comput. Appl.* **35**, 12401–12414 (2023).
23. Liu, Y., Wei, Y., Wang, C. & Wu, H. Trajectory optimization for adaptive deformed wheels to overcome steps using an improved hybrid genetic algorithm and an adaptive particle swarm optimization. *Mathematics* **12**, 2077 (2024).
24. Mokhtari, H. & Hasani, A. An energy-efficient multi-objective optimization for flexible job-shop scheduling problem. *Comput. & Chem. Eng.* **104**, 339–352 (2017).
25. Torabi, S. A., Karimi, B. & Ghomi, S. M. T. F. The common cycle economic lot scheduling in flexible job shops: The finite horizon case. *Int. J. Prod. Econ.* **97**, 52–65 (2005).

26. Gong, G. *et al.* Energy-efficient flexible job shop scheduling problem considering discrete operation sequence flexibility. *Swarm Evol. Comput.* **84**, 101421 (2024).
27. Yue, L. *et al.* Two-stage double deep q-network algorithm considering external non-dominant set for multi-objective dynamic flexible job shop scheduling problems. *Swarm Evol. Comput.* **90**, 101660 (2024).
28. Karaboga, D. & Basturk, B. A powerful and efficient algorithm for numerical function optimization: artificial bee colony (abc) algorithm. *J. Glob. Optim.* **39**, 459–471 (2007).
29. Das, S. & Suganthan, P. N. Differential evolution: A survey of the state-of-the-art. *IEEE Transactions on Evol. Comput.* **15**, 4–31 (2011).
30. Kennedy, J. & Eberhart, R. Particle swarm optimization. In *Proceedings of ICNN'95-international conference on neural networks*, vol. 4, 1942–1948 (IEEE, 1995).
31. Rao, R. V., Savsani, V. J. & Vakharia, D. P. Teaching–learning-based optimization: A novel method for constrained mechanical design optimization problems. *Comput. Des.* **43**, 303–315 (2011).
32. Rao, R. Jaya: A simple and new optimization algorithm for solving constrained and unconstrained optimization problems. *Int. J. Ind. Eng. Comput.* **7**, 19–34 (2016).
33. Mitchell, M. Genetic algorithms: An overview. *Complexity* **1**, 31–39 (1995).
34. Demir, Y. & İşleyen, S. K. Evaluation of mathematical models for flexible job-shop scheduling problems. *Appl. Math. Model.* **37**, 977–988 (2013).
35. Roshanaei, V., Azab, A. & Elmaraghy, H. Mathematical modelling and a meta-heuristic for flexible job shop scheduling. *Int. J. Prod. Res.* **51**, 6247–6274 (2013).
36. Zhou, J., Zhang, Y. & Suganthan, P. N. Constrained large-scale multiobjective optimization based on a competitive and cooperative swarm optimizer. *Swarm Evol. Comput.* **91**, 101735 (2024).
37. Pezzella, F., Morganti, G. & Ciaschetti, G. A genetic algorithm for the flexible job-shop scheduling problem. *Comput. & Oper. Res.* **35**, 3202–3212 (2008).
38. Baykasoğlu, A. & Özbakır, L. Analyzing the effect of dispatching rules on the scheduling performance through grammar based flexible scheduling system. *Int. J. Prod. Econ.* **124**, 369–381 (2010).
39. Nguyen, S., Zhang, M., Johnston, M. & Tan, K. C. Genetic programming for job shop scheduling. *Evol. Swarm Intell. Algorithms* 143–167 (2019).
40. Xiong, H., Wang, H., Shi, S. & Chen, K. Comparison study of dispatching rules and heuristics for online scheduling of single machine scheduling problem with predicted release time jobs. *Expert. Syst. with Appl.* **243**, 122752 (2024).
41. Wang, H., Peng, T., Nassehi, A. & Tang, R. A data-driven simulation-optimization framework for generating priority dispatching rules in dynamic job shop scheduling with uncertainties. *J. Manuf. Syst.* **70**, 288–308 (2023).
42. Bari, P. & Karande, P. Optimal job scheduling to minimize total tardiness by dispatching rules and community evaluation chromosomes. *Decis. Making: Appl. Manag. Eng.* **6**, 201–250 (2023).
43. Tay, J. C. & Ho, N. B. Evolving dispatching rules using genetic programming for solving multi-objective flexible job-shop problems. *Comput. & Ind. Eng.* **54**, 453–473 (2008).
44. Gao, K. Z. *et al.* A two-stage artificial bee colony algorithm scheduling flexible job-shop scheduling problem with new job insertion. *Expert. systems with applications* **42**, 7652–7663 (2015).
45. Jia, S. & Hu, Z.-H. Path-relinking tabu search for the multi-objective flexible job shop scheduling problem. *Comput. & Oper. Res.* **47**, 11–26 (2014).
46. Kemmoe-Tchomte, S., Lamy, D. & Tchernev, N. An effective multi-start multi-level evolutionary local search for the flexible job-shop problem. *Eng. Appl. Artif. Intell.* **62**, 80–95 (2017).
47. Ho, N. B. & Tay, J. C. Solving multiple-objective flexible job shop problems by evolution and local search. *IEEE Transactions on Syst. Man, Cybern. Part C (Applications Rev.)* **38**, 674–685 (2008).
48. Shojaei, A., Jafari, D., Khalag, M. & Dokohaki, P. Developing and solving the multi-objective flexible and sustainable job shop scheduling problem with reverse flow and job rotation considerations in uncertain situations. *J. Optim. Ind. Eng.* **16**, 325–340 (2024).
49. Wu, Z., Fan, H., Sun, Y. & Peng, M. Efficient multi-objective optimization on dynamic flexible job shop scheduling using deep reinforcement learning approach. *Processes* **11**, 2018 (2023).

50. Wang, C., Li, X. & Gao, Y. A novel collaborative evolutionary algorithm with two-population for multi-objective flexible job shop scheduling. *CMES-Computer Model. Eng. & Sci.* **137** (2023).
51. Li, J. *et al.* Flexible job shop scheduling optimization for green manufacturing based on improved multi-objective wolf pack algorithm. *Appl. Sci.* **13**, 8535 (2023).
52. Boejko, W., Uchroński, M. & Wodecki, M. Parallel hybrid metaheuristics for the flexible job shop problem. *Comput. & Ind. Eng.* **59**, 323–333 (2010).
53. Srinath, N., Yilmazlar, I. O., Kurz, M. E. & Taaffe, K. Hybrid multi-objective evolutionary meta-heuristics for a parallel machine scheduling problem with setup times and preferences. *Comput. & Ind. Eng.* **185**, 109675 (2023).
54. Zhang, R., Mao, S., Zhao, S. & Liu, C. An arithmetic optimization algorithm with balanced diversity and convergence for multimodal multiobjective optimization. *Swarm Evol. Comput.* **91**, 101724 (2024).
55. Amirteimoori, A. & Kia, R. Concurrent scheduling of jobs and agvs in a flexible job shop system: a parallel hybrid pso-ga meta-heuristic. *Flex. Serv. Manuf. J.* **35**, 727–753 (2023).
56. Chen, P. & Wang, Q. Learning for multiple purposes: A q-learning enhanced hybrid metaheuristic for parallel drone scheduling traveling salesman problem. *Comput. & Ind. Eng.* **187**, 109851 (2024).
57. Farahmand-Tabar, S. & Ashtari, P. Intelligent cross-entropy optimizer: A novel machine learning-based meta-heuristic for global optimization. *Swarm Evol. Comput.* **91**, 101739 (2024).
58. Amirteimoori, A., Tirkolaee, E. B., Simic, V. & Weber, G.-W. A parallel heuristic for hybrid job shop scheduling problem considering conflict-free agv routing. *Swarm Evol. Comput.* **79**, 101312 (2023).
59. Driss, I., Mouss, K. N. & Laggoun, A. A new genetic algorithm for flexible job-shop scheduling problems. *J. Mech. Sci. Technol.* **29**, 1273–1281 (2015).
60. Nouri, M., Bekrar, A., Jemai, A., Niar, S. & Ammari, A. C. An effective and distributed particle swarm optimization algorithm for flexible job-shop scheduling problem. *J. Intell. Manuf.* **29**, 603–615 (2018).
61. Xu, Y., Wang, L., Wang, S. Y. & Liu, M. An effective teaching–learning-based optimization algorithm for the flexible job-shop scheduling problem with fuzzy processing time. *Neurocomputing* **148**, 260–268 (2015).
62. Yuan, Y. & Xu, H. Flexible job shop scheduling using hybrid differential evolution algorithms. *Comput. & Ind. Eng.* **65**, 246–260 (2013).
63. Xu, Y., Yang, Z., Li, X., Kang, H. & Yang, X. Dynamic opposite learning enhanced teaching–learning-based optimization. *Knowledge-Based Syst.* **188**, 104966 (2020).
64. Zhang, L. *et al.* A novel dynamic opposite learning enhanced jaya optimization method for high efficiency plate–fin heat exchanger design optimization. *Eng. Appl. Artif. Intell.* **119**, 105778 (2023).
65. Rahnamayan, S., Tizhoosh, H. R. & Salama, M. M. Quasi-oppositional differential evolution. In *2007 IEEE congress on evolutionary computation*, 2229–2236 (IEEE, 2007).
66. Li, J. & Meng, Z. Global opposition learning and diversity enhancement based differential evolution with exponential crossover for numerical optimization. *Swarm Evol. Comput.* **87**, 101577 (2024).
67. Li, J., Gao, L. & Li, X. Multi-operator opposition-based learning with the neighborhood structure for numerical optimization problems and its applications. *Swarm Evol. Comput.* **84**, 101457 (2024).
68. Gillard, J. Deterministic global optimization: an introduction to the diagonal approach. *Optim. Methods Softw.* **34**, 218–219 (2019).
69. Rao, R. Jaya: A simple and new optimization algorithm for solving constrained and unconstrained optimization problems. *Int. J. Ind. Eng. Comput.* **7**, 19–34 (2016).
70. Mirjalili, S., Mirjalili, S. M. & Lewis, A. Grey wolf optimizer. *Adv. Eng. Softw.* **69**, 46–61 (2014).
71. Storn, R. & Price, K. Differential evolution – a simple and efficient heuristic for global optimization over continuous spaces. *J. Glob. Optim.* **11**, 341–359 (1997).
72. Paulavicius, R., Sergeyev, Y. D., Kvasov, D. E. & ilinskas, J. Globally-biaseddisimplalgorithm for expensive global optimization. *J. Glob. Optim.* **59**, 545–567 (2014).
73. Janjanam, L., Saha, S. K. & Kar, R. Optimal design of hammerstein cubic spline filter for nonlinear system modeling based on snake optimizer. *IEEE Transactions on Ind. Electron.* **70**, 8457–8467 (2022).

74. Elber, G. A review of a b-spline based volumetric representation: Design, analysis and fabrication of porous and/or heterogeneous geometries. *Comput. Des.* **163**, 103587 (2023).
75. Magome, N., Morita, N., Kaneko, S. & Mitsume, N. Higher-continuity s-version of finite element method with b-spline functions. *J. Comput. Phys.* **497**, 112593 (2024).
76. Wang, C., Xie, Y. M., Zhuang, Z., Zhang, X. & Zhou, S. Strength design of porous materials using b-spline based level set method. *Comput. Methods Appl. Mech. Eng.* **418**, 116490 (2024).
77. Seyala, N. & Abdullah, S. N. Cluster analysis on longitudinal data of patients with kidney dialysis using a smoothing cubic b-spline model. *Int. J. Math. Stat. Comput. Sci.* **2**, 85–95 (2024).
78. Fattahi, P., Mehrabad, M. S. & Jolai, F. Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. *J. Intell. Manuf.* **18**, 331–342 (2007).
79. Aghsami, A. *et al.* A meta-heuristic optimization for a novel mathematical model for minimizing costs and maximizing donor satisfaction in blood supply chains with finite capacity queueing systems. *Healthc. Anal.* **3**, 100136 (2023).
80. Vahedi-Nouri, B., Arbabi, H., Jolai, F., Tavakkoli-Moghaddam, R. & Bozorgi-Amiri, A. Bi-objective collaborative electric vehicle routing problem: Mathematical modeling and matheuristic approach. *J. Ambient Intell. Humaniz. Comput.* **14**, 10277–10297 (2023).
81. Velten, K., Schmidt, D. M. & Kahlen, K. *Mathematical modeling and simulation: introduction for scientists and engineers* (John Wiley & Sons, 2024).
82. Özgüven, C., Özbakır, L. & Yavuz, Y. Mathematical models for job-shop scheduling problems with routing and process plan flexibility. *Appl. Math. Model.* **34**, 1539–1548 (2010).